

RALLE: A Framework for Developing and Evaluating Retrieval-Augmented Large Language Models

Yasuto Hoshi*, Daisuke Miyashita*, Youyang Ng, Kento Tatsuno,
Yasuhiro Morioka, Osamu Torii, Jun Deguchi

Kioxia Corporation, Japan
yasuto1.hoshi@kioxia.com

Abstract

Retrieval-augmented large language models (R-LLMs) combine pre-trained large language models (LLMs) with information retrieval systems to improve the accuracy of factual question-answering. However, current libraries for building R-LLMs provide high-level abstractions without sufficient transparency for evaluating and optimizing prompts within specific inference processes such as retrieval and generation. To address this gap, we present RALLE, an open-source framework designed to facilitate the development, evaluation, and optimization of R-LLMs for knowledge-intensive tasks. With RALLE, developers can easily develop and evaluate R-LLMs, improving hand-crafted prompts, assessing individual inference processes, and objectively measuring overall system performance quantitatively. By leveraging these features, developers can enhance the performance and accuracy of their R-LLMs in knowledge-intensive generation tasks. We open-source our code at <https://github.com/yhoshi3/RaLLe>.

1 Introduction

Large language models (LLMs) have shown great potential for natural language understanding and generation tasks (Brown et al., 2020; Chowdhery et al., 2022; OpenAI, 2023). However, they face challenges when answering factual questions due to hallucinations (or confabulations) (Bang et al., 2023; Borji, 2023), outdated parametric knowledge (Liska et al., 2022), and memory efficiency of parametric knowledge (e.g., Heinzerling and Inui, 2021). To address these limitations, researchers have turned to the retrieval-augmented approach used in open-domain question answering (QA) (Chen et al., 2017), hereinafter referred to as retrieval-augmented LLMs or *R-LLMs*.

In comparison to closed-book settings where language models generate answers without retrieval,

R-LLMs (open-book settings) enable the retrieval of relevant information from external databases or corpora (Mialon et al., 2023; Ng et al., 2023), which has led to improved accuracy in open-domain QA (Shi et al., 2023). Additionally, R-LLMs can acquire extended features even without additional training, such as explicit references, relief from fact hallucination (Nakano et al., 2021), and easy updates to the knowledge source (e.g., Guu et al., 2020; Ng et al., 2023).

Retrieval-augmented generation needs further research and development to reach its full potential. For example, even though the retriever-reader system has been trained on the Natural Questions (NQ) dataset (Kwiatkowski et al., 2019), its F1 score on the short answer task is 68.3 and still lags behind the oracle F1 score of 75.7 (Asai and Choi, 2021). This implies that further improvements can be made to the retrieval-augmented generation approach. Additionally, users would be probably aware that the outputs generated by R-LLMs may contain factual errors, particularly when applied to knowledge-intensive tasks. However, there is currently a lack of accessible evaluation framework to assess their output quality. This makes it difficult to identify areas for improvement.

Furthermore, having effective tools for developing R-LLMs is crucial. These tools should enable the design of inference steps such as retrieve-then-generate, selecting the combination of retrievers and LLMs, evaluating the performance of the entire system, and testing the prompts used in each inference step. Currently available tools, such as the ChatGPT Retrieval Plugin¹, Guidance², and LangChain³ (Chase, 2023), offer a high degree of abstraction, making it challenging to verify the functionality of individual inference steps or opti-

¹<https://github.com/openai/chatgpt-retrieval-plugin>

²<https://github.com/microsoft/guidance>

³Note: Our code does not use either of these.

* These authors contributed equally to this work.

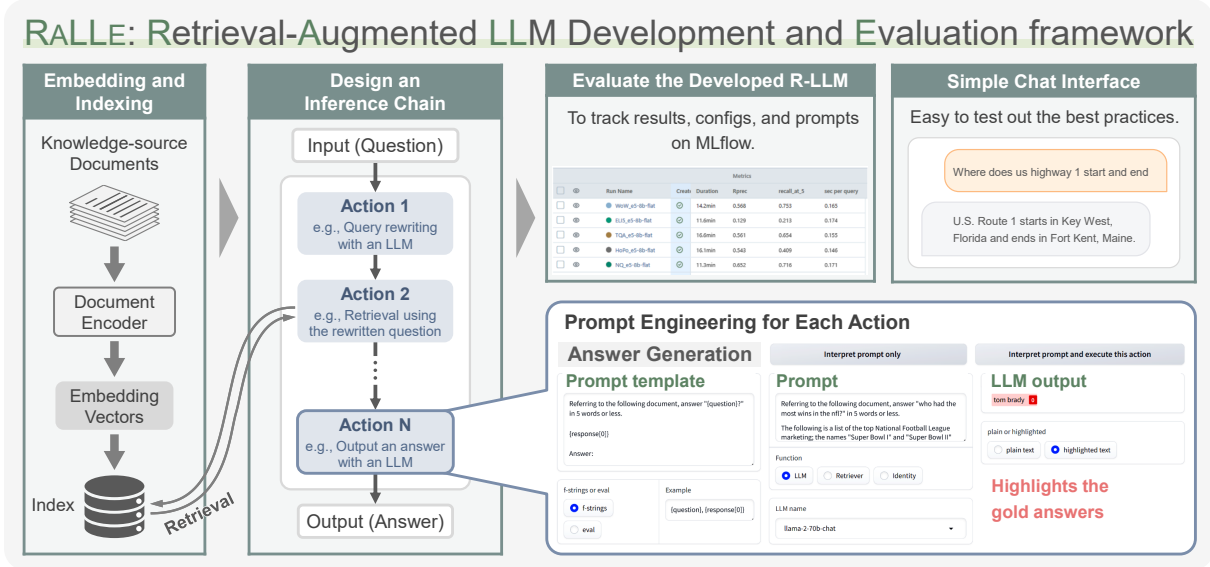


Figure 1: Overview of RALLE, our proposed development and evaluation framework for R-LLMs. Any number of actions can be defined for an R-LLM. Each action can be executed individually to test the corresponding prompts. Experimental setup and evaluation results can be tracked using MLflow. Additionally, a simple chat interface can be built to test out the best practices from the development and evaluation stages in a practical setting.

mize prompts within each step. This lack of transparency might hinder the optimization of R-LLMs.

In this paper, we propose RALLE, an accessible framework for Retrieval-Augmented Large Language model development and Evaluation. We also present evaluation results of several R-LLMs that we have constructed by using open-source retrievers and LLMs. To the best of our knowledge, RALLE is the first framework that empowers R-LLM developers and open-domain QA researchers to efficiently develop, evaluate, and improve R-LLMs using objective metrics.

RALLE offers several key benefits:

1. *Easy development and testing*: users can easily select, combine, and test various retrievers and LLMs, especially open-source models, within a graphical interface.
2. *Objective evaluation of R-LLMs*: RALLE provides reproducible experiments with objective benchmarks/metrics, enabling objective assessments of R-LLM performance.
3. *Transparent prompt engineering*: all inputs (prompts) and outputs of each action are visible to developers, allowing for easy exploration and optimization of the prompts.

2 RALLE Usage

Figure 1 presents an overview of the key features of the proposed framework⁴. The primary development process involves three stages: (1) embedding and indexing the knowledge source documents, (2) designing an inference chain consisting of an R-LLM with customized prompt templates for each action, and (3) benchmarking the developed R-LLM.

2.1 Document Embedding and Indexing

To begin, the knowledge source documents can be encoded using an arbitrary encoder model, such as a sparse or dense retriever. For efficient indexing of dense embeddings, several methods are available by default, including Faiss (Johnson et al., 2019), HNSW (Malkov and Yashunin, 2020), and DiskANN (Jayaram Subramanya et al., 2019). By default, an HNSW index is constructed with $ef_construction = 128$ (the size of the dynamic list for the nearest neighbors) and $m = 32$ (the number of links created for every new element during graph construction).

2.2 Chain Construction

Once the document embedding and indexing are completed, the retrievers (and the corresponding indices) and LLMs can be loaded via the

⁴Please also review the [demonstration screencast](#).

Gradio⁵-based GUI (Abid et al., 2019) to establish an inference chain that comprises an R-LLM. This chain of actions enables users to design a pipeline for multi-step inference, such as *[retrieve]*-*[generate]*, or more intricate workflows such as *[rewrite query]*-*[retrieve]*-*[generate]* proposed in Ma et al. (2023). The versatility of this feature is especially beneficial in creating the chains tailored to specific use cases.

A single-action chain can function as either a simple retriever that returns the retrieved documents, or a *closed-book* QA that leverages the parametric knowledge of an LLM to provide answers without retrieval. In contrast, a chain with multiple actions that include retrieval enables retrieval-augmented generation or *open-book* QA, allowing an LLM to access external documents relevant to a question. Our default setup for R-LLMs consists of two actions: retrieve and generate.

2.3 Prompt Engineering

The RALLE framework allows developers to interactively craft customized prompt templates for LLMs and even for search queries on a per-chain basis. Each action can be executed independently, enabling precise control over LLM responses, such as specifying the desired output format or suppressing undesirable hallucinations. To enhance the versatility of prompt development, RALLE integrates support for f-strings and `eval()` function in Python.

2.4 Experiment Tracking

We utilize MLflow (LF Projects, 2023) to track the experiments, along with their associated configuration files and prompt templates. This allows us to compare the performance of different experiment runs objectively, which enables us to develop even better R-LLMs.

2.5 Chat AI

RALLE also provides support for building a simple chat interface. This enables users to test out best practices from the development and evaluation stages in a practical setting.

3 Experimental Settings

In this section, we evaluate the performance of R-LLMs constructed with several combinations of open-source retrievers and LLMs on knowledge-intensive tasks.

3.1 Tasks and Datasets

We employ KILT (Knowledge Intensive Language Tasks) benchmark (Petroni et al., 2021), an extensive benchmark that encompasses 11 datasets across five knowledge-intensive natural language processing tasks: fact checking, entity linking, slot filling, open-domain question answering, and dialogue (for further details of KILT, see Petroni et al. (2021)). We use the training sets for developing prompts and the development set for evaluation.

As the knowledge source, we utilize the pre-processed Wikipedia passages provided by KILT. The passages are derived from English Wikipedia articles based on the 2019/08/01 Wikipedia dump data, consisting of a total of 5.9 million articles and 22.2 million 100-word passages. For both dense and sparse retrievers, we use the set of 100-word passages after additional pre-processing that prepends the title of the article to each passage.

Note that RALLE is dataset-agnostic, allowing developers to use their own QA datasets and corpora for development and evaluation. See Appendix A.10 for more information.

3.2 Models

This subsection details the retrievers and LLMs employed to build R-LLMs in our experiments. RALLE allows practitioners and researchers to easily experiment with the most recent models available in open-source repositories. With the exception of BM25, all models are available from Hugging Face (Wolf et al., 2020) (see Appendix A.9 for the summary).

3.2.1 LLMs

The LLM used within the R-LLM must comprehend instructions provided in a prompt and generate appropriate responses based on the given information. To achieve this, we use instruction-tuned LLMs with a temperature parameter set to zero for optimal performance and reproducibility.

Llama-2-chat is tuned with supervised fine-tuning and reinforcement learning with human feedback (RLHF) (Christiano et al., 2017; Stiennon et al., 2020) to align to human preferences for helpfulness and safety (Touvron et al., 2023b). In our experiments, we utilize both 13-billion (**Llama2-13B**) and 70-billion (**Llama2-70B**) models.

WizardVicunaLM-13B⁶ (**W-Vicuna-13B**) (Lee, 2023) is formed by combining the concepts

⁵<https://www.gradio.app/>

⁶<https://huggingface.co/junelee/wizard-vicuna-13b>

Model	dim.	max len.	MTEB Retrieval
BM25	-	-	42.3 [♠]
m-e5	1,024	514	51.43
e5	1,024	512	50.56

Table 1: Summary of the retrievers used in our evaluation. Dimensions of a dense embedding vector are shown in *dim.*, while the maximum token length of an input sequence is *max len.*. The evaluation metric for MTEB Retrieval is nDCG@10. [♠]: Results from Ram et al. (2022). Results on MTEB Retrieval except BM25 are copied from MTEB leaderboard⁷.

of WizardLM (Xu et al., 2023) (refining the initial instructions with Evol-Instruct method (Xu et al., 2023)) and Vicuna (Chiang et al., 2023) (a fine-tuned LLaMA model (Touvron et al., 2023a) with multi-round conversation data from chatbots).

3.2.2 Retrievers

We experiment with both sparse and dense retrievers for document retrieval. Specifically, we select dense retrievers that have achieved high accuracy on the retrieval task of Massive Text Embedding Benchmark (MTEB) (Muennighoff et al., 2023) leaderboard⁷ as of July 2023. A list of the retrievers used in our study can be found in Table 1. In the open-book experiments, the top-5 most relevant documents are retrieved.

As the metrics of retrieval performance, we follow Petroni et al. (2021) and use the page-level R-precision (Craswell, 2016) and recall@5. The page-level R-precision is the percentage of R gold pages inside each provenance set among the top- R retrieved pages. Typically, R-Precision is equivalent to Precision@1 except FEVER and HotPotQA (multi-hop datasets).

BM25 (Robertson and Zaragoza, 2009) is a bag-of-words retrieval function based on the term-matching. We use the Pyserini (Lin et al., 2021) implementation of unigram BM25 with the default parameters of $k_1 = 0.9$ (term frequency scaling) and $b = 0.4$ (document length normalization). The documents for BM25 retrieval is the same 100-word passages as the dense retrievers.

e5-large-v2⁸ (e5) (Wang et al., 2022) is a supervised bi-encoder model with a query encoder and a

document encoder. **multilingual-e5-large⁹** (m-e5) is a multilingual fine-tuned e5 model.

3.3 Prompts

We utilize custom-designed prompt templates that are specifically crafted for each dataset in KILT. RALLe accepts templates with non-natural language formats, such as f-strings and eval() functions in Python. This allows developers to carefully craft their prompt templates for optimal performance. The prompt templates used in our experiments are shown in Appendix A.11.

For entity linking task of KILT (AY2, WnWi, and WnCw), we employ a REWRITE-EL template by default for search queries. This template extracts the specific entity mentions being questioned as a query, as employing an entire span of a question is unlikely to find relevant documents (we will discuss in Section 4.3). After retrieving the relevant documents, the top-1 Wikipedia title is output as an answer. As a result, the downstream accuracy in entity linking task is not affected by the number of retrieved documents (if one or more).

4 KILT Benchmark Results

This section provides the downstream and retrieval performance of the R-LLMs developed and evaluated using RALLe.

4.1 Baseline

We compare our results with those of the BART-large model (Lewis et al., 2020a) for the closed-book setting and the RAG model (Lewis et al., 2020b) for the open-book setting, which presented in Petroni et al. (2021). Notably, these baseline models were specifically fine-tuned on the KILT benchmark, whereas our chosen LLMs and constructed R-LLMs were not. See also Appendix A.5 for additional information of the baselines.

4.2 Downstream Performance

We summarize the downstream performance¹⁰ in Table 2. RALLe also includes has_answer percentage for short answers, a proxy metric to measure the proportion of questions that contain gold answers within the final output generated by an R-LLM (see Appendix A.3 for more details).

⁷<https://huggingface.co/spaces/mteb/leaderboard>

⁸<https://huggingface.co/intfloat/e5-large-v2>

⁹<https://huggingface.co/intfloat/multilingual-e5-large>

¹⁰See also Table 6 in Appendix A.6 for additional results in a closed-book setting.

	Fact Check.	Entity Linking			Slot Filling		Open Domain QA				Dial.
Dataset	FEV	AY2	WnWi	WnCw	T-REx	zsRE	NQ	HoPo	TQA	ELI5	WoW
Model / Metric	Accuracy						Exact Match			RL	F1
BART-large [◇] (<i>closed-book</i>)	<u>80.7</u>	86.6	47.9	48.0	<u>43.8</u>	3.0	26.2	16.9	32.5	<u>22.7</u>	13.8
Llama2-70B (<i>closed-book</i>)	33.6 (74.9)	39.8 (54.5)	42.8 (53.8)	39.2 (55.7)	28.5 (40.5)	11.3 (13.6)	19.6 (37.4)	13.9 (25.1)	67.4 (80.8)	23.0	<u>13.3</u>
RAG [◇]	87.7	<u>77.4</u>	49.0	<u>46.7</u>	61.5	47.4	48.8	<u>27.7</u>	61.7	16.1	<u>13.3</u>
e5 + W-Vicuna-13B	10.6 (42.4)	51.2 (57.9)	<u>48.6</u> (51.4)	45.6 (51.4)	31.6 (46.1)	23.0 (29.3)	18.7 (38.0)	19.7 (28.3)	43.1 (67.7)	21.4	12.3
e5 + Llama2-13B	66.3 (73.5)	<u>51.2</u> (57.9)	48.6 (51.4)	<u>45.6</u> (51.4)	17.2 (42.3)	31.7 (41.1)	36.1 (43.3)	14.3 (25.5)	56.3 (76.2)	20.9	12.3
BM25 + Llama2-70B	46.2 (86.3)	18.0 (35.9)	19.1 (32.2)	14.2 (30.9)	25.9 (43.0)	31.4 (37.8)	25.3 (34.3)	25.9 (33.4)	65.8 (80.0)	21.3	12.2
e5 + Llama2-70B	49.9 (88.6)	<u>51.2</u> (57.9)	48.6 (51.4)	<u>45.6</u> (51.4)	28.9 (49.2)	35.0 (43.2)	<u>36.4</u> (48.8)	28.1 (35.8)	<u>71.1</u> (83.9)	21.5	13.2
e5 (DiskANN)	49.9 (87.9)	44.3 (50.5)	45.3 (48.1)	43.0 (48.8)	25.3 (43.9)	32.1 (37.9)	36.1 (48.4)	26.7 (34.3)	70.4 (83.2)	21.5	13.1
top-2	49.3 (88.1)	51.2 (57.9)	48.6 (51.4)	<u>45.6</u> (51.4)	23.5 (44.9)	34.7 (43.0)	33.7 (46.2)	23.8 (34.2)	71.3 (82.9)	21.6	<u>13.3</u>
top-10	50.2 (88.0)	<u>51.2</u> (57.9)	48.6 (51.4)	<u>45.6</u> (51.4)	31.1 (49.3)	<u>35.4</u> (42.5)	35.2 (48.1)	24.9 (35.7)	59.3 (82.8)	21.5	13.2
Model / Metric	KILT-Accuracy						KILT-EM		KILT-RL	KILT-F1	
RAG [◇]	55.5	77.4	49.0	46.7	25.4	42.6	36.3	3.1	36.1	2.7	7.5
e5 + W-Vicuna-13B	8.4 (33.5)	<u>51.2</u> (51.2)	<u>48.6</u> (48.6)	<u>45.5</u> (45.5)	19.0 (28.0)	22.2 (28.1)	14.4 (27.8)	8.6 (11.9)	26.6 (40.3)	2.7	7.3
e5 + Llama2-13B	<u>53.1</u> (58.7)	51.2 (51.2)	48.6 (48.6)	45.5 (45.5)	11.5 (25.7)	29.8 (38.5)	27.5 (32.5)	5.6 (10.6)	34.7 (46.1)	2.7	7.4
BM25 + Llama2-70B	21.9 (44.4)	17.6 (17.6)	18.9 (18.9)	13.9 (13.9)	14.5 (22.5)	24.9 (29.6)	9.3 (12.4)	4.5 (5.9)	23.6 (27.9)	<u>1.5</u>	4.0
e5 + Llama2-70B	40.2 (71.2)	51.2 (51.2)	48.6 (48.6)	<u>45.5</u> (45.5)	19.2 (29.7)	32.8 (40.4)	<u>27.7</u> (36.3)	11.3 (14.5)	<u>42.8</u> (49.7)	2.7	<u>8.1</u>
e5 (DiskANN)	38.3 (68.5)	44.3 (44.3)	45.3 (45.3)	42.8 (42.8)	19.3 (24.2)	30.2 (35.5)	27.3 (35.9)	9.3 (12.1)	42.1 (49.0)	2.7	8.0
top-2	39.6 (70.7)	51.2 (51.2)	48.6 (48.6)	<u>45.5</u> (45.5)	15.6 (28.0)	32.9 (40.6)	25.7 (35.2)	7.6 (13.1)	43.1 (49.3)	2.7	8.3
top-10	40.4 (70.7)	51.2 (51.2)	48.6 (48.6)	<u>45.5</u> (45.5)	<u>20.5</u> (29.9)	<u>33.2</u> (39.8)	27.1 (36.1)	<u>9.9</u> (14.3)	36.1 (48.9)	2.7	<u>8.1</u>

Table 2: Downstream performance on KILT dev set. Following [Petroni et al. \(2021\)](#), we report the results of typical metrics for each dataset, with bold indicating the best result and underlined indicating the second. The metrics with the prefix *KILT*- award output performance only when R-Prec = 1 (retrieval success). The figures in parentheses represent has_answer percentage, which corresponds to the proportion of questions with gold answers included in the final output. The figures shown in gray are copied from the column above because they do not change based on the given setting (we use the Identity function of RALLE for the tasks, rather than an LLM). [◇]: Results from [Petroni et al. \(2021\)](#).

Our constructed R-LLM (e5 + Llama2-70B) surpasses the performance of the RAG model on both HoPo and TQA, despite not being fine-tuned with KILT like RAG. Moreover, our constructed R-LLMs demonstrate acceptable accuracy levels on other datasets as well, without any significant drawbacks. The results indicate that the LLMs used in this study exhibit certain ability to comprehend the retrieved documents.

Furthermore, our analysis reveals several factors that could contribute to improvement of downstream performance, including retrieval augmentation (except ELI5), increased model scale (except FEV and T-REx), and referring to more documents during generation (except NQ, HoPo, TQA and WoW). However, some datasets exhibits exceptions to these tendencies or had lower performance compared to their corresponding has_answer percentage (such as FEV, T-REx, NQ, and TQA). To address this issue, developers can improve the R-LLM with RALLE by refining the inference chain and the prompt templates. In Section A.4, we provide our initial attempts at developing inference

chains with three actions on several datasets.

Overall, the downstream evaluation results provide valuable insights into how well the constructed R-LLMs perform on knowledge-intensive tasks, enabling developers to identify areas for improvement.

4.3 Retrieval Performance

Table 3 shows retrieval performance of the chosen retrievers on KILT development set (see also Table 8 in Appendix for the results of recall@5). According to Table 3, e5 (with Faiss Flat index) achieves the highest retrieval performance on average, though m-e5 is better on MTEB Retrieval task (Table 1). Despite the superior retrieval accuracy of e5 compared to RAG on KILT, the downstream performance of the R-LLM which employs e5 falls short of that of RAG (Table 2). This indicates that there is potential room for improvement through further optimized prompts to enhance the performance on a target dataset.

As described in Section 3.3, REWRITE-EL serves as the default template for search queries

	Fact Check.	Entity Linking			Slot Filling		Open Domain QA				Dial.	
<i>Dataset</i>	FEV	AY2	WnWi	WnCw	T-REx	zsRE	NQ	HoPo	TQA	ELI5	WoW	Avg.
<i>Model</i>	R-Precision											
RAG [◇]	63.5	77.4	49.0	46.7	29.3	65.4	60.3	30.8	49.3	16.4	46.7	48.6
BM25	52.1	17.7	20.6	15.3	34.0	57.7	26.3	41.3	31.7	6.8	28.8	30.2
− REWRITE-EL	52.1	3.0 (−14.7)	0.1 (−20.5)	2.8* (−12.5)	34.0	57.7	26.3	41.3	31.7	6.8	28.8	25.9 (−4.3)
m-e5 (Flat)	81.7	41.8	45.8	41.6	47.1	81.4	63.0	54.0	56.1	11.9	57.9	52.9
− REWRITE-EL	81.7	3.2 (−38.6)	0.1 (−45.7)	3.1 (−38.5)	47.1	81.4	63.0	54.0	56.1	11.9	57.9	41.8 (−11.1)
e5 (Flat)	82.0	51.6	51.6	49.2	45.3	81.9	65.2	54.3	56.1	12.9	56.8	55.2
− REWRITE-EL	82.0	3.4 (−48.2)	0.0 (−51.6)	2.6 (−46.6)	45.3	81.9	65.2	54.3	56.1	12.9	56.8	41.9 (−13.3)
e5 (HNSW)	67.9	38.9	42.3	40.5	23.1	53.0	60.3	34.9	50.4	10.2	54.5	43.3
− REWRITE-EL	67.9	2.9 (−36.0)	0.0 (−42.3)	1.6 (−38.9)	23.1	53.0	60.3	34.9	50.4	10.2	54.5	32.6 (−10.7)
e5 (DiskANN)	78.8	44.7	47.8	46.0	37.1	74.5	64.9	49.1	55.4	12.9	56.6	51.6
− REWRITE-EL	78.8	3.2 (−41.5)	0.1 (−47.7)	1.8 (−44.2)	37.1	74.5	64.9	49.1	55.4	12.9	56.6	39.4 (−12.2)

Table 3: Retrieval performances on KILT dev set. We report page-level R-Precision on KILT development set. Avg. refers to macro-average of the retrieval scores in each dataset. Bold indicates the best result. [◇]: Results from [Petroni et al. \(2021\)](#). *: BM25 (without REWRITE-EL) failed with long queries (45 out of 5,599 questions) in WnCw.

<i>Retrieval</i>			
Model	Avg. R-Prec	Memory	sec/Q
BM25	30.2	-	0.121
e5 (Flat)	55.2	84.8 GB	0.169
e5 (HNSW)	43.3	90.4 GB	0.008
e5 (DiskANN)	51.6	10.9 GB	0.022
<i>Completion in the Closed-Book Setting</i>			sec/Q
Llama-70B			6.727
<i>Retrieval + Generation</i>			sec/Q
BM25 + Llama2-70B			3.637
e5 + Llama2-70B			3.793
e5 (DiskANN) + Llama2-70B			3.628

Table 4: Execution latency in seconds per question (sec/Q). *Memory* in Retrieval indicates the maximum (DRAM) memory footprints.

related to entity linking task (AY2, WnWi, and WnCw). As shown in Table 3, employing the REWRITE-EL template leads to higher retrieval accuracy when compared to using the full question text as a search query (− REWRITE-EL setting). This indicates that omitting unnecessary information from the search queries is helpful especially for entity linking task.

4.4 Speed Analysis

RALLE allows users to optimize the trade-off between latency (in seconds per question) and accuracy by comparing various configurations. As demonstrated in Table 4, employing approximate nearest neighbor search (ANNS) algorithms such as HNSW and DiskANN can significantly reduce

retrieval latency at the cost of decreased accuracy. Note that, the optimal balance between speed and accuracy depends on the specific requirements of the application, and RALLE enables users to easily experiment with diverse ANNS settings to determine their impact on both factors.

Notably, DiskANN achieves an accuracy that is only slightly lower than Faiss flat index while significantly improving search speeds, despite requiring less memory footprints than both flat and HNSW indices. Though the reduction in R-LLM execution time achieved through ANNS may appear relatively minor, the significantly lower DRAM requirements of DiskANN could make it a more practical solution for scenarios where DRAM capacity is limited and the flat index exceeds available DRAM capacity. For further details regarding latency, refer to Table 9 in Appendix A.8.

5 Conclusion

This paper introduces RALLE, an accessible framework for developing and evaluating R-LLMs. We also report evaluation results of several R-LLMs built using open-source retrievers and LLMs on knowledge-intensive tasks. Overall, RALLE offers a significant advancement in retrieval-augmented generation research, enabling efficient development, evaluation, and improvement of R-LLMs. We hope that RALLE will contribute to the development of best practices for R-LLMs.

Limitations

All KILT evaluations presented in this paper were conducted using a development set to maintain fair-

ness and consistency across evaluations, as the answers of the test set remain confidential¹¹.

While R-LLMs exhibit high validity, it falls behind the smaller yet specialized model, RAG, on the KILT downstream task (refer to Table 2). This disparity can be attributed to various factors, including prompt maturity and the ability of LLMs to generate responses. Although the employed prompts were carefully developed, it is likely that more optimal prompts exist (discussed in Section 4.3). Moreover, fine-tuning LLMs with retrieval-augmented generation tasks might enhance their performance on downstream tasks. Therefore, the evaluation accuracy reported herein would represent a conservative estimate.

Prompt engineering is a crucial aspect of the retrieval-augmented generation process, as the generated outputs can differ significantly between models, even when provided with the same prompt. RALLE offers an advantage in this regard, allowing users to effortlessly experiment with diverse prompts for varying behaviors, datasets, and intricate chain of actions.

In the realm of prompt development, techniques like Automatic Prompt Engineer (APE) (Zhou et al., 2023) automate the creation of prompts from input-output pairs and sampling to identify the most effective prompts. However, the input-output pairs in retrieval-augmented generation are distinctly different from those of the simple instruction induction tasks. Because the input text for retrieval-augmented generation can often be lengthy and complex, it is difficult to automatically induce the effective prompts from the input-output pairs.

This tool enables developers to construct an inference chain with predefined actions, while recent advances have also introduced methods allowing LLMs to determine the actions (Yao et al., 2023). One approach entails retrieving documents using a query rewritten by an LLM and then summarizing them until the desired information is obtained. However, in our initial experiments (not described in this paper), we observed instances where relatively small LLMs (typically less than 100 billion parameters) became trapped in cycles of repeated retrieval and summarization, hindering their ability to reach the final answer generation. Our tool addresses this issue by intentionally building explicit inference chains to avoid unintended operations.

¹¹<https://eval.ai/web/challenges/challenge-page/689/overview>

References

- Abubakar Abid, Ali Abdalla, Ali Abid, Dawood Khan, Abdulrahman Alfozan, and James Zou. 2019. [Gradio: Hassle-free sharing and testing of ml models in the wild](#). *arXiv preprint arXiv:1906.02569*.
- Akari Asai and Eunsol Choi. 2021. [Challenges in information-seeking QA: Unanswerable questions and paragraph retrieval](#). In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 1492–1504, Online. Association for Computational Linguistics.
- Yejin Bang, Samuel Cahyawijaya, Nayeon Lee, Wenliang Dai, Dan Su, Bryan Wilie, Holy Lovenia, Ziwei Ji, Tiezheng Yu, Willy Chung, et al. 2023. [A multi-task, multilingual, multimodal evaluation of chatgpt on reasoning, hallucination, and interactivity](#). *arXiv preprint arXiv:2302.04023*.
- Ali Borji. 2023. [A categorical archive of ChatGPT failures](#). *arXiv preprint arXiv:2302.03494*.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. [Language models are few-shot learners](#). In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc.
- Harrison Chase. 2023. LangChain. <https://langchain.com/>.
- Danqi Chen, Adam Fisch, Jason Weston, and Antoine Bordes. 2017. [Reading Wikipedia to answer open-domain questions](#). In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1870–1879, Vancouver, Canada. Association for Computational Linguistics.
- Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. 2023. [Vicuna: An open-source chatbot impressing GPT-4 with 90%* ChatGPT quality](#).
- Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben

- Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayanan Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. 2022. [PaLM: Scaling language modeling with pathways](#). *arxiv:2204.02311*.
- Paul F Christiano, Jan Leike, Tom Brown, Miljan Martić, Shane Legg, and Dario Amodei. 2017. [Deep reinforcement learning from human preferences](#). In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc.
- Nick Craswell. 2016. *R-Precision*, pages 1–1. Springer New York, New York, NY.
- Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. 2020. [Retrieval augmented language model pre-training](#). In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 3929–3938. PMLR.
- Benjamin Heinzerling and Kentaro Inui. 2021. [Language models as knowledge bases: On entity representations, storage capacity, and paraphrased queries](#). In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pages 1772–1791, Online. Association for Computational Linguistics.
- Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. [Diskann: Fast accurate billion-point nearest neighbor search on a single node](#). In *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc.
- Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. [Billion-scale similarity search with GPUs](#). *IEEE Transactions on Big Data*, 7(3):535–547.
- Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. [Dense passage retrieval for open-domain question answering](#). In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, Online. Association for Computational Linguistics.
- Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. 2019. [Natural questions: A benchmark for question answering research](#). *Transactions of the Association for Computational Linguistics*, 7:452–466.
- June Lee. 2023. WizardVicunaLM. <https://github.com/melodysdreamj/WizardVicunaLM>.
- Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. 2020a. [BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension](#). In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7871–7880, Online. Association for Computational Linguistics.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020b. [Retrieval-augmented generation for knowledge-intensive nlp tasks](#). In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc.
- LF Projects. 2023. MLflow – a platform for the machine learning lifecycle. <https://mlflow.org/>.
- Jimmy Lin, Xueguang Ma, Sheng-Chieh Lin, Jheng-Hong Yang, Ronak Pradeep, and Rodrigo Nogueira. 2021. [Pyserini: A Python toolkit for reproducible information retrieval research with sparse and dense representations](#). In *Proceedings of the 44th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 2021)*, pages 2356–2362.
- Adam Liska, Tomas Kocisky, Elena Gribovskaya, Tayfun Terzi, Eren Sezener, Devang Agrawal, Cyprien De Masson D’Autume, Tim Scholtes, Manzil Zaheer, Susannah Young, Ellen Gilsenan-Mcmahon, Sophia Austin, Phil Blunsom, and Angeliki Lazaridou. 2022. [StreamingQA: A benchmark for adaptation to new knowledge over time in question answering models](#). In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 13604–13622. PMLR.
- Xinbei Ma, Yeyun Gong, Pengcheng He, Hai Zhao, and Nan Duan. 2023. [Query rewriting for retrieval-augmented large language models](#). *arXiv preprint arXiv:2305.14283*.
- Yu A. Malkov and D. A. Yashunin. 2020. [Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs](#). *IEEE Trans. Pattern Anal. Mach. Intell.*, 42(4):824–836.
- Grégoire Mialon, Roberto Dessì, Maria Lomeli, Christoforos Nalmpantis, Ram Pasunuru, Roberta Raileanu, Baptiste Rozière, Timo Schick, Jane Dwivedi-Yu,

- Asli Celikyilmaz, Edouard Grave, Yann LeCun, and Thomas Scialom. 2023. [Augmented language models: a survey](#). *arXiv preprint arXiv:2302.07842*.
- Niklas Muennighoff, Nouamane Tazi, Loic Magne, and Nils Reimers. 2023. [MTEB: Massive text embedding benchmark](#). In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, pages 2014–2037, Dubrovnik, Croatia. Association for Computational Linguistics.
- Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. 2021. [Webgpt: Browser-assisted question-answering with human feedback](#). *arXiv preprint arXiv:2112.09332*.
- Youyang Ng, Daisuke Miyashita, Yasuto Hoshi, Yasuhiro Morioka, Osamu Torii, Tomoya Kodama, and Jun Deguchi. 2023. [SimplyRetrieve: A private and lightweight retrieval-centric generative ai tool](#). *arXiv preprint arXiv:2308.03983*.
- OpenAI. 2023. [GPT-4 technical report](#). *arXiv preprint arXiv:2303.08774*, abs/2303.08774.
- Fabio Petroni, Aleksandra Piktus, Angela Fan, Patrick Lewis, Majid Yazdani, Nicola De Cao, James Thorne, Yacine Jernite, Vladimir Karpukhin, Jean Maillard, Vassilis Plachouras, Tim Rocktäschel, and Sebastian Riedel. 2021. [KILT: a benchmark for knowledge intensive language tasks](#). In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 2523–2544, Online. Association for Computational Linguistics.
- Ori Ram, Liat Bezael, Adi Zicher, Yonatan Belinkov, Jonathan Berant, and Amir Globerson. 2022. [What are you token about? dense retrieval as distributions over the vocabulary](#). *arXiv preprint arXiv:2212.10380*.
- Stephen Robertson and Hugo Zaragoza. 2009. [The probabilistic relevance framework: BM25 and beyond](#). *Found. Trends Inf. Retr.*, 3(4):333–389.
- Weijia Shi, Sewon Min, Michihiro Yasunaga, Minjoon Seo, Rich James, Mike Lewis, Luke Zettlemoyer, and Wen-tau Yih. 2023. [RePlug: Retrieval-augmented black-box language models](#). *arXiv preprint arXiv:2301.12652*.
- Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F Christiano. 2020. [Learning to summarize with human feedback](#). In *Advances in Neural Information Processing Systems*, volume 33, pages 3008–3021. Curran Associates, Inc.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023a. [LLaMA: Open and efficient foundation language models](#). *arXiv preprint arXiv:2302.13971*.
- Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023b. [Llama 2: Open foundation and fine-tuned chat models](#). *arXiv preprint arXiv:2307.09288*.
- Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. 2022. [Text embeddings by weakly-supervised contrastive pre-training](#). *arXiv preprint arXiv:2212.03533*.
- Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander Rush. 2020. [Transformers: State-of-the-art natural language processing](#). In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online. Association for Computational Linguistics.
- Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, and Daxin Jiang. 2023. [WizardLM: Empowering large language models to follow complex instructions](#). *arXiv preprint arXiv:2304.12244*.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2023. [ReAct: Synergizing reasoning and acting in language models](#). In *The Eleventh International Conference on Learning Representations*.
- Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. 2023. [Large language models are human-level prompt engineers](#). In *The Eleventh International Conference on Learning Representations*.

A Appendix

A.1 Computational Resources

The evaluation experiments are conducted on an Ubuntu 20.04.6 server equipped with Intel(R) Xeon(R) Gold 6326 CPU at 2.90 GHz CPU cores, and one node with 4×NVIDIA A100 Tensor Core GPU with 40 GB memory, and a RAID-5 array with a Dell(R) PERC H745 Front controller and KIOXIA(R) PM6-R SAS SSDs for storage. The CUDA version is 12.2, the Python version is 3.9.16, the PyTorch version is 2.0.1, and the Transformers version is 4.29.2.

Load models
Develop chain
Chat
Config

Dataset
NQ

Question
who had the most wins in the nfl

Config
Load config
Chain length
2
Execute entire chain

Action 1

Interpret prompt only
Interpret prompt and execute this action

prompt_template[0]
{question}

f-strings or eval
f-strings
eval

Example
{question}, {response[0]}

prompt[0]
who had the most wins in the nfl

Function
LLM
Retriever
Identity

Retriever name
e5-8b_flat

k, press ENTER key to apply
5

response[0]
following is list of top national football league nfl quarterbacks in wins in nfl quarterback is only position that is credited with records of wins and losses active quarterback tom brady holds records for most wins with 237 most regular season wins wi

wiki_id_title[0]
List of National Football League career quarterback wins leaders s, 13929036 ; Super Bowl, 27718; List of Super Bowl champions, 2145410; List of National Football League head coaches with 50 wins, 41018355; List of National Football League career quarterback wins leaders, 13929036

Action 2

Interpret prompt only
Interpret prompt and execute this action

prompt_template[1]
Referring to the following document, answer "{question}" in 5 words or less.
{response[0]}
Answer:

f-strings or eval
f-strings
eval

Example
{question}, {response[0]}

prompt[1]
Referring to the following document, answer "who had the most wins in the nfl?" in 5 words or less.
The following is a list of the top National Football League (NFL) quarterbacks in wins. In the NFL, the quarterback is the only position that is credited with records of wins and losses. Active quarterback Tom Brady holds the records for most wins with 237, most regular season wins with 207, and marketing; the names "Super Bowl I" and "Super Bowl II"

Function
LLM
Retriever
Identity

LLM name
llama-2-70b-chat

response[1]
tom brady

plain or highlighted
plain text
highlighted text

show qa data

```

{
  qa_data:
  {
    id: "5289242154789678439",
    input: "who had the most wins in the nfl",
    output: [
      0: {
        answer: "Tom Brady",
        provenance: [
          0: {
            wikipedia_id: "13929036",
            title: "List of National Football League career quarterback wins leaders",
            start_paragraph_id: 2,
            start_character: 19,
            end_paragraph_id: 2,
            end_character: 28,
            bleu_score: 1,
            section: "Section:::Abstract."
          }
        ]
      }
    ]
  }
}

```

Figure 2: A screenshot of the *Development chain* tab of RALLE. Developers can create tailored action chains comprising multiple actions of inference. For each action, developers can specify a prompt template, confirm the results of applying the template, and execute the action using the newly defined prompt, individually. Moreover, RALLE can highlight the gold answers within the retrieved documents or the output of the LLM, as well as highlight the Wikipedia IDs of successfully retrieved provenance.

A.2 Development Screen of RALLE

Figure 2 shows the chain development screen¹². Developers can create an inference chain for an R-LLM on this *Develop chain* tab. One can choose a dataset and specify the desired chain length, which represents the total number of actions. By default, there are two actions: retrieving with a retriever and generating with an LLM.

Prompt templates for each action can be defined using f-strings or eval functions in Python. The results of applying the template can be confirmed without executing retrieval and generation. The execution result can be viewed by clicking the *Interpret prompt and execute this action* button.

The available action operators are *LLM*, *Retriever*, and *Identity*. *LLM* generates text based on the given prompt. *Retriever* retrieves the top k most relevant documents related to the input query. And *Identity* simply outputs the original prompt without employing a retriever or an LLM.

To execute the entire chain, click the *Execute entire chain* button. At the bottom of this tab, the selected question and its corresponding answer can be reviewed. Also, RALLE enables to highlight the gold answers within the retrieved documents or the output of the LLM, as well as highlight the Wikipedia ID of successfully retrieved provenance.

A.3 Additional Metric: has_answer

RALLE also includes has_answer percentage (e.g., Karpukhin et al., 2020) for short answers, a proxy metric to measure the proportion of questions that contain gold answers within the final output generated by an R-LLM. By tracking this metric, developers can identify situations where the model generates responses that include gold answers but may be overlooked due to evaluation biases such as exact matching. This information can help refine prompts to improve overall performance.

A.4 Attempts to Build 3-action Chain

According to Section 4.2, retrieval augmentation has a significant impact on performance in fact checking, open-domain QA for short answers, and slot filling tasks when comparing the closed-book and open-book settings of Llama2-70B. In entity linking task (AY2, WnWi, and WnCw), however, our approach described in Section 3.3 (retrieve, then output the top-1 retrieved Wikipedia title) may not be effective.

To improve the performance, we construct a 3-action chain for AY2 dataset: (1) retrieve top-5 relevant documents, (2) explain the entity mention being questioned, and (3) predict the Wikipedia title based on the explanation and top-5 retrieved titles. Additionally, we explore developing 3-action chains for T-REx and NQ datasets, which involves (1) retrieval, (2) question rewriting, and (3) answer generation. Table 12 shows the prompts used in 3-action chains.

Table 5 shows the downstream performances with the 3-action chains on AY2, NQ, and T-REx datasets. While the 3-action chain outperforms the 2-action (retrieve-then-generate) chain on NQ dataset, it underperforms the 2-action accuracies on AY2 and T-REx datasets. This suggests that the 3-action chains constructed specifically for these two datasets require further optimization. However, the has_answer value for AY2 (70.0%) is higher than that of the 2-action chain (47.8%), indicating that incorporating post-processing steps into the 3-action chain (thus to be 4-action chain) could potentially boost accuracy, particularly for AY2.

One of the benefits of our tool is that it allows for easy definition of such additional inference actions. This means that developers can customize the chain to perform specific tasks beyond the default setting, giving them greater flexibility and control over their development.

A.5 Details of Baseline Model in Open-Book Setting

As a baseline in open-book setting, we present the results of the Retrieval-Augmented Generation (RAG) model (Lewis et al., 2020b) shown in Petroni et al. (2021), which achieved strong performance in the KILT benchmark. The RAG model comprises a bi-encoder retriever and a sequence-to-sequence generator (BART model (Lewis et al., 2020a)), both of which are trained end-to-end. The total number of trainable parameters in the RAG model is approximately 626 million. It is important to note that the RAG model was trained specifically for the KILT benchmark, whereas our chosen LLMs and constructed R-LLMs were not.

A.6 KILT Downstream Performances in Closed-Book Setting

Table 6 summarizes the KILT downstream results in a closed-book setting. The baseline (BART-large) model has been fine-tuned on the KILT

¹²Please also review the [demonstration screencast](#).

Dataset	Fact Check.	Entity Linking			Slot Filling		Open Domain QA				Dial.
	FEV	AY2	WnWi	WnCw	T-REx	zsRE	NQ	HoPo	TQA	ELI5	WoW
Model / Metric	Accuracy						Exact Match			RL	F1
Llama2-70B (<i>closed-book</i>)	33.6 (74.9)	39.8 (54.5)	42.8 (53.8)	39.2 (55.7)	28.5 (40.5)	11.3 (13.6)	19.6 (37.4)	13.9 (25.1)	67.4 (80.8)	23.0	13.3
RAG $\diamond$	87.7	77.4	49.0	46.7	61.5	47.4	48.8	27.7	61.7	16.1	13.3
e5 + Llama2-70B	49.9 (88.6)	51.2 (57.9)	48.6 (51.4)	45.6 (51.4)	28.9 (49.2)	35.0 (43.2)	36.4 (48.8)	28.1 (35.8)	71.1 (83.9)	21.5	13.2
3-action	-	24.4 (70.0)	-	-	16.3 (46.8)	-	36.9 (49.3)	-	-	-	-
Model / Metric	KILT-Accuracy						KILT-EM		KILT-RL		KILT-F1
RAG $\diamond$	55.5	77.4	49.0	46.7	25.4	42.6	36.3	3.1	36.1	2.7	7.5
e5 + Llama2-70B	40.2 (71.2)	51.2 (51.2)	48.6 (48.6)	45.5 (45.5)	19.2 (29.7)	32.8 (40.4)	27.7 (36.3)	11.3 (14.5)	42.8 (49.7)	2.7	8.1
3-action	-	9.5 (27.7)	-	-	10.4 (27.9)	-	28.0 (36.6)	-	-	-	-

Table 5: Downstream performance of the 3-action chain on KILT dev set along with baselines. The figures in parentheses represent has_answer percentage, which corresponds to the proportion of questions with gold answers included in the final output of the LLM. $\diamond$: Results from Petroni et al. (2021).

Dataset	Fact Check.	Entity Linking			Slot Filling		Open Domain QA				Dial.
	FEV	AY2	WnWi	WnCw	T-REx	zsRE	NQ	HoPo	TQA	ELI5	WoW
Model / Metric	Accuracy						Exact Match			RL	F1
BART-large $\diamond$	80.7	86.6	47.9	48.0	43.8	3.0	26.2	16.9	32.5	22.7	13.8
W-Vicuna-13B	0.0 (58.4)	0.1 (52.2)	2.0 (44.9)	0.0 (48.1)	17.9 (33.0)	5.9 (8.5)	6.2 (27.4)	1.7 (17.1)	20.0 (64.5)	22.7	12.7
Llama2-13B	26.3 (50.7)	34.6 (47.5)	35.0 (42.8)	28.5 (41.3)	26.9 (36.7)	7.8 (9.9)	11.5 (29.1)	8.3 (20.3)	43.0 (70.2)	27.6	13.0
Llama2-70B	33.6 (74.9)	39.8 (54.5)	42.8 (53.8)	39.2 (55.7)	28.5 (40.5)	11.3 (13.6)	19.6 (37.4)	13.9 (25.1)	67.4 (80.8)	23.0	13.3

Table 6: Downstream performance on KILT development set in a *closed-book* setting (generation without retrieval). Following Petroni et al. (2021), we report the results of typical metrics for each dataset, with bold indicating the best result. The figures in parentheses represent has_answer percentage, which corresponds to the proportion of questions with gold answers included in the final output of the LLM. $\diamond$: Results from Petroni et al. (2021).

datasets, while our chosen LLMs have not. Despite this, the LLMs demonstrate superior performance compared to the baseline on several datasets.

Specifically, the Llama2-70B model outperforms the BART baseline on the zsRE and TQA datasets, and the Llama2-13B model outperforms the baseline on the ELI5 dataset. This suggests that the parametric knowledge embedded in the LLMs and their capacity for text generation can be leveraged effectively for knowledge-intensive tasks, even zero-shot setting. Nevertheless, as described in Section 4.2, retrieval augmentation can enhance the performance on downstream tasks, except the ELI5 dataset. We also present the closed-book performances of several LLMs on the development set of NQ dataset in Table 7.

A.7 Additional Results for Retrieval Performance

Table 8 presents the recall@5 of the retrievers used in our experiments. Note that even though m-e5 outperforms e5 on the MTEB Retrieval task (shown in Table 1), e5 still demonstrates superior performance compared to m-e5 in terms of both

R-precision (shown in Table 3) and recall@5.

A.8 Details of Speed Analysis

Table 9 presents the details of speed analysis on KILT development set. The search speed of BM25 (without REWRITE-EL) decreases as the total number of words in a query increases. In contrast, for dense vector search, the search speed remains relatively constant regardless of the size of the query due to the fixed dimensionality of the embedding vectors.

According to Table 9, the execution times required for generation with an LLM is longer than the times required for retrieval, particularly when generating lengthy responses such as ELI5 and WoW. Therefore, it may seem counterintuitive that the advantages of ANNS used in vector search are not fully realized in terms of execution time of R-LLMs. However, as previously discussed in Section 4.4, DiskANN requires less memory compared to other vector search algorithms, which means that using such algorithm can actually help conserve computational resources for R-LLM.

We observe that Llama2-13B requires more time

<i>NQ</i>				
Model Name	EM	has_answer	f1	sec/Q
Llama-2-70b-chat	19.6	37.4	36.8	2.254
Llama-2-13b-chat	11.5	29.1	28.1	1.179
StableBeluga2	16.2	40.9	35.5	2.858
gpt-3.5-turbo	25.4	38.9	41.1	-

Table 7: Accuracies on NQ dev set in a closed-book setting. For gpt-3.5-turbo (version 0613), the accuracy was calculated excluding five questions out of 2,837 questions in the NQ development set that were deemed inappropriate prompts by OpenAI and were not processed.

	Fact Check.	Entity Linking			Slot Filling		Open Domain QA				Dial.	
<i>Dataset</i>	FEV	AY2	WnWi	WnCw	T-REx	zsRE	NQ	HoPo	TQA	ELI5	WoW	Avg.
<i>Model</i>	Recall@5											
RAG [◇]	76.1	77.5	49.0	46.7	33.7	73.1	65.5	12.3	56.9	27.3	66.6	53.1
BM25	74.2	28.8	34.7	30.6	42.7	74.7	42.5	22.8	48.7	12.3	45.1	41.6
– REWRITE-EL	74.2	7.6 (−21.2)	3.1 (−31.6)	5.9* (−24.7)	42.7	74.7	42.5	22.8	48.7	12.3	45.1	34.5 (−7.1)
m-e5 (Flat)	91.0	58.5	60.6	62.2	53.1	87.0	69.5	40.4	65.4	19.1	75.0	62.0
– REWRITE-EL	91.0	7.8 (−50.7)	3.8 (−56.8)	5.5 (−56.7)	53.1	87.0	69.5	40.4	65.4	19.1	75.0	47.1 (−14.9)
m-e5 (HNSW) – REWRITE-EL	63.2	4.9	3.5	2.6	26.0	48.2	55.6	14.1	48.7	14.6	66.8	31.7
e5 (Flat)	90.6	66.1	63.3	66.7	52.1	87.2	71.6	40.9	65.4	21.3	75.3	63.7
– REWRITE-EL	90.6	7.6 (−58.5)	3.4 (−59.9)	4.8 (−61.9)	52.1	87.2	71.6	40.9	65.4	21.3	75.3	47.3 (−16.4)
e5 (HNSW)	74.7	49.6	50.7	50.8	26.7	55.9	65.2	19.3	58.6	16.0	70.9	48.9
– REWRITE-EL	74.7	6.0 (−43.6)	3.3 (−47.4)	3.3 (−47.5)	26.7	55.9	65.2	19.3	58.6	16.0	70.9	36.4 (−12.5)
e5 (DiskANN)	86.6	57.1	58.3	60.9	42.1	78.7	70.7	34.7	64.6	20.8	75.0	59.0
– REWRITE-EL	86.6	7.4 (−49.7)	3.3 (−55.0)	3.6 (−57.3)	42.1	78.7	70.7	34.7	64.6	20.8	75.0	44.3 (−14.7)

Table 8: Retrieval performances (recall@5) on KILT dev set. Avg. refers to macro-average of the scores in each dataset. Bold indicates the best result. The figures shown in gray are copied from the column above because they do not change based on the given setting. [◇]: Results from Petroni et al. (2021). *: BM25 (without REWRITE-EL) failed with long queries (45 out of 5,599 questions) in WnCw.

to process each question compared to Llama2-70B. Upon further analysis, we discovered that the Llama2-13B model occasionally produced non-sensical responses such as multiple newline characters (“\n”), partially due to the limitations of our prompts.

A.9 Model Information

As shown in Table 10, we utilize several open-source models from Hugging Face, specifically their officially released versions. We load the distributed models in 8-bit precision by default except Llama2-70B model (in 4-bit) using Hugging Face Accelerate¹³ library.

A.10 Using Custom Datasets

In addition to utilizing KILT datasets, RALLIE enables developers to develop and evaluate R-LLMs on their own QA datasets and corpora. To use the

custom datasets with RALLIE, you will need to perform the following preprocessing:

- Prepare your corpus as a TSV file containing the document IDs, texts, and titles.
- Create a JSONL file for your QA dataset. The format should look like this: {"id": "", "input": "", "output": [{"answer": "", "provenance": [{"wikipedia_id": "", "title": ""}]}]}, where “input” represents a question.

See our repo for more detailed instructions: <https://github.com/yhoshi3/RaLLie>.

A.11 The Prompts used in the Evaluation

Table 11 summarizes the prompts used in our experiment. *Open-book* indicates retrieve-then-generate setting. The queries used for retrieval are the raw questions without any rewriting, except for the REWRITE-EL settings of AY2, WnWi, and WnCw.

¹³<https://huggingface.co/docs/accelerate/index>

	Fact Check.	Entity Linking			Slot Filling		Open Domain QA				Dial.	
Tasks	FEV	AY2	WnWi	WnCw	T-REx	zsRE	NQ	HoPo	TQA	ELI5	WoW	Avg.
Models	Completion in Closed-Book Setting (in seconds per question)											
W-Vicuna-13B	1.565	13.040	10.870	9.793	0.983	1.142	2.165	1.969	1.414	22.820	7.122	6.626
Llama2-13B	0.625	1.077	1.036	1.201	0.940	0.913	1.270	1.185	1.014	40.100	9.522	5.353
Llama2-70B	1.765	2.936	2.745	2.618	1.953	2.031	2.285	2.188	1.877	42.500	11.100	6.727
Retrieval + Generation (in seconds per question)												
e5 + W-Vicuna-13B	1.529	1.310	1.368	1.158	1.192	1.453	2.595	1.945	1.734	15.480	10.850	3.692
e5 + Llama2-13B	1.084	1.165	1.209	1.046	1.300	1.407	1.284	1.975	9.830	32.48	16.76	6.322
BM25 + Llama2-70B	1.841	0.008	0.009	0.008	2.015	2.296	2.206	2.344	2.249	15.020	12.010	3.637
e5 + Llama2-70B	1.926	0.133	0.131	0.135	2.135	2.424	2.419	2.346	2.238	16.030	11.810	3.793
e5 (top-2) + Llama2-70B	1.544	0.133	0.131	0.135	1.661	1.908	1.994	1.833	1.759	15.120	10.820	3.367
e5 (top-10) + Llama2-70B	2.811	0.133	0.131	0.135	2.951	3.276	-	13.900	14.400	35.070	24.100	-
e5 (DiskANN) + Llama2-70B	1.803	0.044	0.044	0.043	2.009	2.281	2.166	2.247	2.116	15.780	11.370	3.628
e5 + Llama2-70B (3-action)	-	25.41	-	-	4.993	-	16.320	-	-	-	-	-
Retrieval (in seconds per question)												
BM25	0.038	0.008	0.009	0.008	0.018	0.013	0.052	0.105	0.086	0.136	0.857	0.121
BM25 (without REWRITE-EL)	0.038	5.700	4.531	5.440	0.018	0.013	0.052	0.105	0.086	0.136	0.857	1.543
m-e5 (Flat)	0.174	0.164	0.166	0.176	0.187	0.165	0.194	0.156	0.176	0.177	0.165	0.173
m-e5 (HNSW)	0.008	0.013	0.013	0.015	0.008	0.009	0.009	0.009	0.009	0.011	0.010	0.010
e5 (Flat)	0.177	0.168	0.172	0.159	0.201	0.170	0.171	0.146	0.155	0.174	0.165	0.169
e5 (HNSW)	0.008	0.008	0.008	0.008	0.008	0.008	0.008	0.008	0.008	0.008	0.009	0.008
e5 (DiskANN)	0.018	0.020	0.038	0.020	0.020	0.030	0.020	0.021	0.019	0.019	0.021	0.022
Mean Query Length (tokens)												
	11.1±4.0	357.9±149.0	331.5±113.5	505.2±31.1	7.5±2.5	7.6±2.3	9.9±2.1	19.5±6.6	17.9±8.9	21.0±10.7	86.3±58.0	

Table 9: Execution time (in seconds per question) in RALLÉ. Avg. refers to macro-average of the times in each task. The mean query length and its standard deviation (shown as $\pm$ after the value) are also displayed, which were calculated using the e5 tokenizer.

Language Model				
Model Name	Size	max len.	emb dim	URL
wizard-vicuna-13b (Lee, 2023)	13,015,864,320	2,048	-	https://huggingface.co/junelee/wizard-vicuna-13b
Llama-2-13b-chat (Touvron et al., 2023b)	13,015,864,320	4,096	-	https://huggingface.co/meta-llama/Llama-2-13b-chat
Llama-2-70b-chat (Touvron et al., 2023b)	68,976,653,312	4,096	-	https://huggingface.co/meta-llama/Llama-2-70b-chat
StableBeluga2	70B	4,096	-	https://huggingface.co/stabilityai/StableBeluga2
Retriever				
multilingual-e5-large	559,890,946	514	1,024	https://huggingface.co/intfloat/multilingual-e5-large
e5-large-v2 (Wang et al., 2022)	335,142,400	512	1,024	https://huggingface.co/intfloat/e5-large-v2

Table 10: Hugging Face links of the models used in our evaluation. *Size* refers to the total number of effective parameters of each model. *max len.* refers to the maximum token length of model input.

Closed-book indicates that an LLM answers to the given question without retrieval. Although these prompts have been our established best practices, we recognize that there may be opportunities for improvement (see also Section 5).

Open-book	Closed-book
FEVER ☒ f-strings ☐ eval	
Action 1: Retriever <pre>{question}</pre> Action 2: LLM <pre>{response[0]} ↵ Answer IN ONE WORD if the document SUPPORTS or REFUTES "{question}". ↵ Answer:</pre>	Action 1: LLM <pre>Answer IN ONE WORD if your knowledge SUPPORTS or REFUTES "{question}". ↵ Answer:</pre>
AY2 ☐ f-strings ☒ eval	
Action 1: Retriever <pre>'What is "' + {}'.format(question).split('[START_ENT']')[1].split('[END_ENT'])[0][1:-1] + "' ?'</pre> Action 2: Identity <pre>'{}'.format(wiki_id_title[0]).split('; ')[0].split(', ')[0]</pre>	Action 1: LLM <pre>'What is the most relevant Wikipedia title to the en- tity "' + {}'.format(question).split('[START_ENT] ')[1].split('[END_ENT'])[0] + "' in the context of "' + {}'.format(question).split('[START_ENT'])[0][-100:] + {}'.format(question).split('[START_ENT'])[1].split('[END_ENT'])[0] + {}'.format(question).split('[END_ENT'])[1][:100] + "'...?"\n\nPlease answer only the Wikipedia title.\n\nAnswer: ''</pre>
WnWi ☐ f-strings ☒ eval	
Action 1: Retriever <pre>'What is "' + {}'.format(question).split('[START_ENT']')[1].split('[END_ENT'])[0][1:-1] + "' ?'</pre> Action 2: Identity <pre>'{}'.format(wiki_id_title[0]).split('; ')[0].split(', ')[0]</pre>	Action 1: LLM <pre>'What is the most relevant Wikipedia title to the en- tity "' + {}'.format(question).split('[START_ENT] ')[1].split('[END_ENT'])[0] + "' in the context of "' + {}'.format(question).split('[START_ENT'])[0][-100:] + {}'.format(question).split('[START_ENT'])[1].split('[END_ENT'])[0] + {}'.format(question).split('[END_ENT'])[1][:100] + "'...?"\n\nPlease answer only the Wikipedia title.\n\nAnswer: ''</pre>
WnCw ☐ f-strings ☒ eval	
Action 1: Retriever <pre>'What is "' + {}'.format(question).split('[START_ENT']')[1].split('[END_ENT'])[0][1:-1] + "' ?'</pre> Action 2: Identity <pre>'{}'.format(wiki_id_title[0]).split('; ')[0].split(', ')[0]</pre>	Action 1: LLM <pre>'What is the most relevant Wikipedia title to the en- tity "' + {}'.format(question).split('[START_ENT] ')[1].split('[END_ENT'])[0] + "' in the context of "' + {}'.format(question).split('[START_ENT'])[0][-100:] + {}'.format(question).split('[START_ENT'])[1].split('[END_ENT'])[0] + {}'.format(question).split('[END_ENT'])[1][:100] + "'...?"\n\nPlease answer only the Wikipedia title.\n\nAnswer: ''</pre>

Continued on next page...

Table 11: Prompt templates used in our experiments. The hook-left arrows ↵ refers to new line. Note that RALLÉ supports f-strings and eval() function in Python.

Table 11 – continued from previous page.

Open-book	Closed-book
<i>T-REx</i>	
Action 1: Retriever (f-strings) <pre>{question}</pre> Action 2: LLM (eval()) <pre>'''Referring to the following document, answer "what is the ''' + '{}'.format(question).split('[SEP]')[1] + 'of ' + '{}'.format(question).split('[SEP]')[0] + ' '? in 5 words or less.\n\n''' + '{}'.format(response[0]) + '\n\n''' + '{}'.format(question).split('[SEP]')[1] + '':</pre>	Action 1: LLM (eval()) <pre>'What is the ' + ''' + '{}'.for- mat(question).split('[SEP] ')[1] + ' of ' + '{}'.for- mat(question).split('[SEP]')[0] + ' in 5 words or less?\n\n''' + '{}'.format(question).split('[SEP] ')[1] + ': '</pre>
<i>zsRE</i>	
Action 1: Retriever (f-strings) <pre>{question}</pre> Action 2: LLM (eval()) <pre>Referring to the following document, answer "{ques- tion}?" in 5 words or less. ← ← {response[0]} ← ← Answer:</pre>	Action 1: LLM (eval()) <pre>'Tell me the ' + ''' + '{}'.for- mat(question).split('[SEP] ')[1] + ' of ' + '{}'.for- mat(question).split('[SEP]')[0] + ' in 5 words or less.\n\n''' + '{}'.format(question).split('[SEP] ')[1] + ': '</pre>
<i>NQ</i> ☒ f-strings ☐ eval	
Action 1: Retriever <pre>{question}</pre> Action 2: LLM <pre>Referring to the following document, answer "{ques- tion}?" in 5 words or less. ← ← {response[0]} ← ← Answer:</pre>	Action 1: LLM <pre>Answer '{question}?' in 5 words or less. ← ← Answer:</pre>
<i>HoPo</i> ☒ f-strings ☐ eval	
Action 1: Retriever <pre>{question}</pre> Action 2: LLM <pre>Referring to the following document, answer "{ques- tion}?" in 5 words or less. ← ← {response[0]} ← ← Answer:</pre>	Action 1: LLM <pre>Answer '{question}?' in 5 words or less. ← ← Answer:</pre>

Continued on next page...

Table 11 – continued from previous page.

Open-book	Closed-book
TQA ☒ f-strings ☐ eval	
Action 1: Retriever {question} Action 2: LLM Referring to the following document, answer "{question}?" in 5 words or less. {response[0]} Answer:	Action 1: LLM Answer '{question}' in 5 words or less. Answer:
ELI5 ☒ f-strings ☐ eval	
Action 1: Retriever {question} Action 2: LLM Referring to the following document, answer "{question}". {response[0]} Explain the following questions as if I were five years old. {question} Answer:	Action 1: LLM Explain '{question}' as if I were five years old. Answer:
WoW ☒ f-strings ☐ eval	
Action 1: Retriever {question} Action 2: LLM Referring to the following document, output a short and informative reply to the conversation. {response[0]} Referring to the above document, output a short and informative reply to the following conversation. This conversation ends on your turn. {question} Informative and short answer:	Action 1: LLM Output a short and informative reply to the conversation. This conversation ends on your turn. {question} Informative and short answer:

AY2

☐ f-strings

☒ eval

Action 1: Retriever

```
'What is "' + '{}'.format(question).split(' [START_ENT] ')[1].split(' [END_ENT]')[0] + '" in the context of "' +
'{}'.format(question).split(' [START_ENT]')[0][-100:] + '{}'.format(question).split(' [START_ENT]')[1].split(' [END_ENT]')[0]
+ '{}'.format(question).split(' [END_ENT]')[1][:100] + '...?'
```

Action 2: LLM

```
'What is "' + '{}'.format(question).split(' [START_ENT] ')[1].split(' [END_ENT]')[0] + '" in the context of "' +
'{}'.format(question).split(' [START_ENT]')[0][-100:] + '{}'.format(question).split(' [START_ENT]')[1].split(' [END_ENT]')[0]
+ '{}'.format(question).split(' [END_ENT]')[1][:100] + '...'?
Answer in a short and conc sentence.' + '\n\nAnswer:\n''
```

Action 3: LLM

```
'Please select the most appropriate title for the word "' + '{}'.format(question).split(' [START_ENT]
')[1].split(' [END_ENT]')[0] + '" based on the given Description.' + '\n\nIf none of these titles suit your needs, please
suggest a possible alternative title.' + '\n\nTitles: \n' + '/ '.join([titleid.split(',')[0] for titleid in '{}'.for-
mat(wiki_id_title[0]).split('; ')]) + '\n\nDescription:\n' + '{}'.format(response[1]) + '\n\nWikipedia Title:\n''
```

T-REx

☒ f-strings

☐ eval

Action 1: Retriever

```
{question}
```

Action 2: LLM

```
Formulate a question that asks [SEP] in the following sentence: ←
'{question}' ←
Generated question:
```

Action 3: LLM

```
{response[0]} ←
Referring to the document above, answer "{response[1]}" in 5 words or less. ←
Answer:
```

NQ

☒ f-strings

☐ eval

Action 1: Retriever

```
{question}
```

Action 2: LLM

```
Please rewrite the following question clearly. ←
{question}? ←
Rewritten question:
```

Action 3: LLM

```
Referring to the following document, answer "{response[1]}" in 5 words or less. ←
{response[0]} ←
Answer:
```

Table 12: Prompt templates used in 3-action chains.